

Project Executable Files

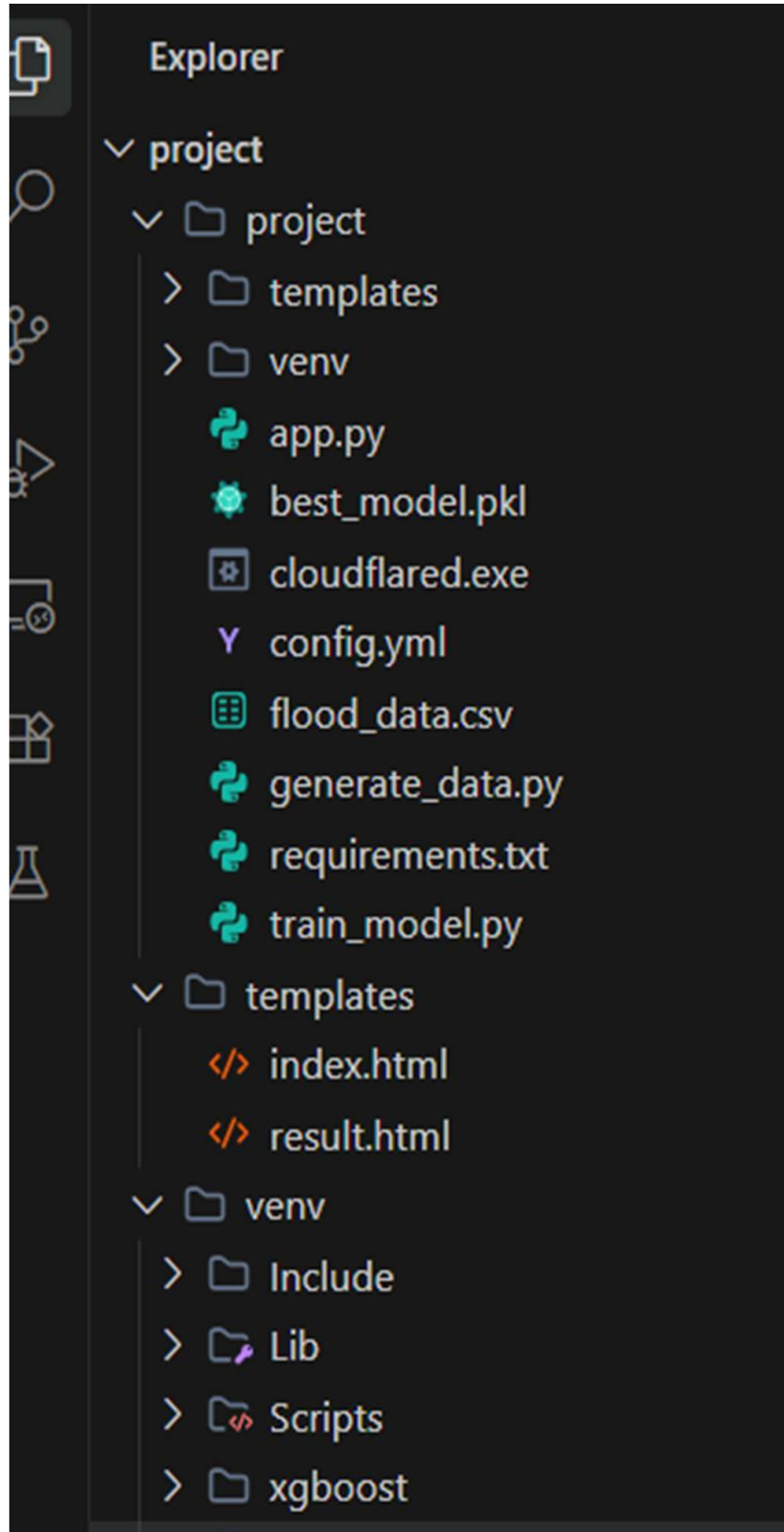
Date	03 july 2026
Team ID	
Project Name	Rising Waters
Maximum Marks	3 Marks

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	NA
5	Environment configuration file (.env.example or similar)	No
6	Deployed application URL (if hosted)	No
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	No
9	Test files and test results	Yes

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:



Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://rising-waters-flood-prediction-1-5a8x.onrender.com
Login Credentials (Demo)	NA
Platform / Hosting Provider	<i>Render</i>
Repository Link	https://github.com/venkatesh09060444/Rising-Waters-Flood-Prediction
Demo Video Link	https://drive.google.com/file/d/10v5I06fNcZU0rUsE94udETDqIInd6bmS_/view?usp=drivesdk

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Step 4: Run Instructions

1. Download or clone the **Flood Prediction** project from the GitHub repository.
2. Open the project folder in **Visual Studio Code** or any Python IDE.
3. Install the required dependencies using:

```
</> Bash
```

```
pip install -r requirements.txt
```

4. Start the Flask application by running:

```
</> Bash
```

```
python app.py
```

5. Open a web browser and visit:

```
http://127.0.0.1:5000
```

6. Enter the required weather parameters (such as annual rainfall, cloud visibility, and seasonal rainfall).
7. Click the **Predict** button to generate the flood prediction result.
8. The application will display the predicted flood risk based on the trained **XGBoost** model.

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	The system uses historical weather data and does not support real-time weather data.	Future versions will integrate real-time weather APIs for improved prediction accuracy.
2	The application is currently deployed on a local Flask server, limiting scalability.	Planned deployment on IBM Cloud to support more users and improve availability.
3	The system does not provide automatic SMS or email alerts after prediction.	Future enhancement includes notification services for early flood warnings.